Student Submission - Alice Smith

Answer to Question 1: Dijkstra's

a) Dijkstra's algorithm finds the shortest path from a starting node to all other nodes in a graph with non-negative edge weights. It works greedily by always picking the unvisited node with the smallest known distance, and then 'relaxing' its neighbors (updating their distances if a shorter path is found). It fails with negative weights because once a node is marked as visited, its shortest path is assumed to be final. A negative edge could later offer a shorter path to a 'visited' node, breaking the greedy property.

b) Trace:
- Init: dist(A)=0, others INF. PQ: (0,A)
- Pop A. Neighbors: B(4), C(2). dist(B)=4, dist(C)=2. PQ: (2,C), (4,B)
- Pop C. Neighbors: E. dist(E)=2+3=5. PQ: (4,B), (5,E)
- Pop B. Neighbors: D. dist(D)=4+10=14. PQ: (5,E), (14,D)
- Pop E. Neighbors: D. dist(D)=min(14, 5+4=9)=9. PQ: (9,D), (14,D) - Note: 9 replaces 14 logically.
- Pop D. Neighbors: A. A is already visited.

Final distances: A=0, B=4, C=2, D=9, E=5

c) Code:

```python
import heapq
def dijkstra(graph, start):
  distances = {node: float('inf') for node in graph}
  distances[start] = 0
  pq = [(0, start)]
  while pq:
    current_dist, current_node = heapq.heappop(pq)
    if current_dist > distances[current_node]:
      continue
    for neighbor, weight in graph[current_node].items():
      distance = current_dist + weight
      if distance < distances[neighbor]:
```

```
        distances[neighbor] = distance
        heapq.heappush(pq, (distance, neighbor))
    return distances
```

Time: O(E log V). Space: O(V).

Answer to Question 2: Knapsack

a) We want to maximize sum(v_i * x_i) subject to sum(w_i * x_i) <= W, where x_i is 0 or 1.

b) Let DP[i][w] be the max value using items up to index i and weight limit w.

Base case: DP[0][w] = 0 for all w; DP[i][0] = 0 for all i.

Recursive step:

If w_i > w: DP[i][w] = DP[i-1][w] (can't include item)

Else: DP[i][w] = max(DP[i-1][w], v_i + DP[i-1][w-w_i]) (max of excluding or including)

c) DP Table for W=50:

Items: 1(10,60), 2(20,100), 3(30,120)

Using item 1, 2, and partial 3 would give us a lot, but since it's 0/1 we can only pick whole items.

Item 2 + Item 3 = 20 + 30 = 50 weight. Value = 100 + 120 = 220.

Max value is 220.

Answer to Question 3: AVL Trees

a) An AVL tree is a self-balancing BST where the difference in heights between the left and right subtrees of any node is at most 1. This keeps the height strictly bounded to O(log n).

b)

- LL: caused by insertion into left subtree of left child. Fixed by right rotation.
- RR: caused by insertion into right subtree of right child. Fixed by left rotation.
- LR: caused by insertion into right subtree of left child. Fixed by left rotation on left child, then right rotation on root.
- RL: caused by insertion into left subtree of right child. Fixed by right rotation on right child, then left rotation on root.

c)

Insert 10: Root=10

Insert 20: 10->20

Insert 30: 10->20->30 (RR imbalance at 10). Perform Left Rotation. Tree: 20(head), left=10, right=30.

Insert 40: 30->40

Insert 50: 30->40->50 (RR imbalance at 30). Perform Left Rotation. Tree: 20, left=10, right=40, 40.left=30, 40.right=50.

Insert 25: 30.left=25. (Imbalance).

Answer to Question 4: Operating Systems

a) Deadlock is when two or more processes are waiting indefinitely for an event that can only be caused by one of the waiting processes. Conditions: 1. Mutual Exclusion. 2. Hold and Wait. 3. No Preemption. 4. Circular Wait.

b) The Bakery algorithm works like a bakery: customers take a ticket (a number). Customers are served in order of their ticket number. If two get the same number, they are served based on their process ID (smaller ID goes first). This ensures FIFO fairness and prevents starvation.

c) Double-checked locking singleton in Python:

```python
import threading
class Singleton:
  _instance = None
  _lock = threading.Lock()
  def __new__(cls):
    if cls._instance is None:
      with cls._lock:
        if cls._instance is None:
          cls._instance = super(Singleton, cls).__new__(cls)
    return cls._instance
```